News from PREEMPT_RT

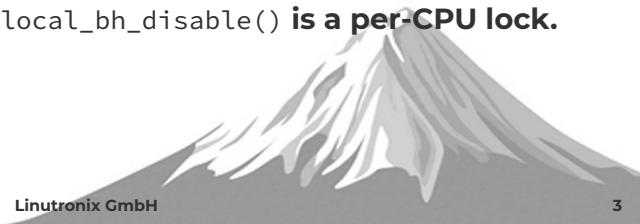




- Unlocked local-BH
- Futex private hash
- Lazy preempt follow-up
- Workqueue changes
- Documentation

Softirq on PREEMPT_RT vs vanilla

- Preemptible. Context switch is possible.
- Runs after the threaded handler.
- No piggyback after hardirq.
- Everything from hardirq goes to ksoftirqd.
- Due to disabled preemption `local_bh_disable()` is a per-CPU lock.



Results of the lock

- Ressources depending on BH locking are protected. 👍
- Long–running forced threaded interrupts block other forced threaded interrupts. 👎
- Timer, Tasklets, ...block forced threaded interrupts. 👎
- Best we can do is a PI–boost. The need–resched condition is never observed. 👎



Trace force-threaded interrupts preempted

```
irq/40-eno0-2034 D...2  681 softirq_raise: vec=3 [action=NET_RX]
irq/40-eno0-2034 ..s.2  681 softirq_entry: vec=3 [action=NET_RX]
irq/40-eno0-2034 d.H.3  690 irq_handler_entry: irq=35
irq/40-eno0-2034 dNH33  692 sched_wakeup: irq/35-ahci prio=44
irq/40-eno0-2034 d.s23  694 sched_switch: prio=49 R+->irq/35-ahci prio=44

irq/35-ahci-837  d..31  696 sched_pi_setprio: irq/40-eno0 prio 49 -> 44
irq/35-ahci-837  d..21  699 sched_switch: prio=44 D->irq/40-eno0 prio=44

irq/40-eno0-2034 d.s34  715 sched_wakeup: iperf3 prio=120
irq/40-eno0-2034 d..21  736 sched_switch: prio=49 R+->irq/35-ahci prio=44

irq/35-ahci-837  D..13  740 softirq_raise: vec=4 [action=BLOCK]
irq/35-ahci-837  ..s.2  740 softirq_entry: vec=4 [action=BLOCK]
```

But why exactly the lock

- A FPGA interrupt handler doing only wake up of userland.
- A CAN driver needs to inject packet.
- AHCI driver needs to do IO.
- local_bh_disable() and enable() plus the lock:
  - Preserve the raise softirq and invoke semantic.
  - Protect per-CPU ressources. Mostly.



What happend

- **Proposed the concept of explicit serialisation for BH**
- **Happend in 2023 on Plumbers to network maintainers**
- **Implemented on top of** `local_lock()` **as** `local_lock_nested_bh()`
- **Used mostly in networking**
- `CONFIG_PREEMPT_RT_NEEDS_BH_LOCK` **available since v6.18**



Trace force-threaded interrupts preempted

```
irq/38-eno0-2006 D...1  032 softirq_raise: vec=3 [action=NET_RX]
irq/38-eno0-2006 ..s.1  032 softirq_entry: vec=3 [action=NET_RX]
irq/38-eno0-2006 d.H.1  033 irq_handler_entry: irq=35 name=ahci
irq/38-eno0-2006 dNH31  034 sched_wakeup: irq/35-ahci prio=44
irq/38-eno0-2006 d.s21  035 sched_switch: prio=49 R+->irq/35-ahci prio=44

irq/35-ahci-842  D..12  038 softirq_raise: vec=4 [action=BLOCK]
irq/35-ahci-842  ..s.1  039 softirq_entry: vec=4 [action=BLOCK]
irq/35-ahci-842  ..s.1  042 softirq_exit: vec=4 [action=BLOCK]
irq/35-ahci-842  d..2.  043 sched_switch: prio=44 S->irq/38-eno0 prio=49

irq/38-eno0-2006 ..s.1  044 softirq_exit: vec=3 [action=NET_RX]
irq/38-eno0-2006 d..2.  051 sched_switch: prio=49 S->swapper/2 prio=120
```

local_lock_nested_bh example

```
DEFINE_PER_CPU(struct page_pool_bh, system_page_pool) = {
        .bh_lock = INIT_LOCAL_LOCK(bh_lock),
};

static int netif_skb_check_for_xdp(struct sk_buff **pskb, ...)
{

    ...
    local_lock_nested_bh(&system_page_pool.bh_lock);
    err = skb_cow_data_for_xdp(this_cpu_read(system_page_pool.pool), ...
    local_unlock_nested_bh(&system_page_pool.bh_lock);

    ...
```

Alternative approach

```
struct task_struct {
 ...
    struct bpf_run_ctx   *bpf_ctx;
 ...
}
do_xdp_generic(const struct bpf_prog *xdp_prog, struct sk_buff **pskb){
    struct bpf_net_context __bpf_net_ctx, *bpf_net_ctx;

    bpf_net_ctx = bpf_net_ctx_set(&__bpf_net_ctx);
    /* XDP code */
    bpf_net_ctx_clear(bpf_net_ctx);
}
```

According to Ricardo Cañuelo Navarro's syzbot instance, this fixed a use-after-free. See commit.

FUTEX

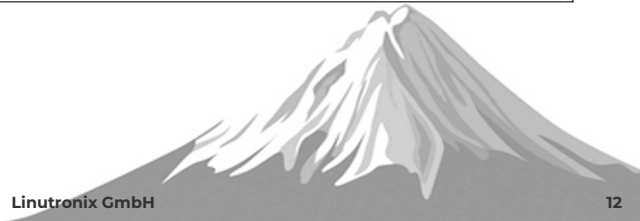
- pthread_mutex_t lock
- pthread_cond_t cond
- pthread_mutex_lock(&lock);
- pthread_mutex_unlock(&lock);
- pthread_cond_wait(&cond, &lock);
- pthread_cond_signal(&cond);



Locking example

```
pthread_mutex_t  lock

            Thread0                                          Thread1
pthread_mutex_lock(&lock);
                                                  pthread_mutex_lock(&lock);
```



A bit more verbose locking

```
typedef struct pthread_mutex {
    unsigned int lock;
    ....
} pthread_mutex_t

int pthread_mutex_lock(struct pthread_mutex *m)
{
    do {
        val = m->lock;
        if (!val)
            break;
        sys_futex(&m->lock, FUTEX_WAIT | FUTEX_PRIVATE_FLAG, val);
        } while (1);
    ...
}
```

A bit more detailed

```
int pthread_mutex_lock(struct pthread_mutex *m)
{ do { val = m->lock;
  if (!val) {
    oldval = atomic_compare_exchange(&m->lock, 0, LOCKED);
    if (oldval == 0) return 0;
    continue;
  }
  if (val == LOCKED) {
    oldval = atomic_compare_exchange(&m->lock, LOCKED, LOCKED | WAITER);
    if (oldval != LOCKED)
      continue;
   }
   sys_futex(&m->lock, FUTEX_WAIT | FUTEX_PRIVATE_FLAG, LOCKED | WAITER);
   } while (1);
}
```

The simple unlocking side

```
int pthread_mutex_unlock(struct pthread_mutex *m)
{
  val = m->lock;
  if (val == LOCKED) {
    oldval = atomic_compare_exchange(&m->lock, LOCKED, 0);
    if (oldval == LOCKED) return 0;
  }
  atomic_compare_exchange(&m->lock, LOCKED | WAITER, 0);
  sys_futex(&m->lock, FUTEX_WAKE | FUTEX_PRIVATE_FLAG, 2);
  return 0;
}
```



Kernel's view

- User space takes of the locking logic
- Kernel is used for sleeping
- If locking was successfull, kernel has no clue
- Only on contention there is a list of waiters
- LOCK_PI and UNLOCK_PI are similar in principle.
- It only gets a uaddr for a waiter



Kernel's view

```
union futex_key {
  struct {
   struct mm_struct *mm;
   unsigned long address;
   unsigned int offset;
  } private;
};
```



Kernel's view

```
union futex_key {
  struct {
   struct mm_struct *mm;
   unsigned long address;
   unsigned int offset;
  } private;
};
```

```
struct futex_q {
    struct plist_node list;
    struct task_struct *task;
    union futex_key key;
};
```



Kernel's view

```
union futex_key {
  struct {
   struct mm_struct *mm;
   unsigned long address;
   unsigned int offset;
  } private;
};

struct futex_hash_bucket {
  atomic_t waiters;
  spinlock_t lock;
  struct plist_head chain;
};
```

```
struct futex_q {
    struct plist_node list;
    struct task_struct *task;
    union futex_key key;
};
```



Kernel's view

```
union futex_key {
  struct {
   struct mm_struct *mm;
   unsigned long address;
   unsigned int offset;
  } private;
};

struct futex_hash_bucket {
  atomic_t waiters;
  spinlock_t lock;
  struct plist_head chain;
};
```

```
struct futex_q {
    struct plist_node list;
    struct task_struct *task;
    union futex_key key;
};

static struct {
  struct futex_hash_bucket *queues;
  unsigned long hashmask;
} __futex_data;
```

How is it bad for PREEMPT_RT?

- TaskA and TaskB are not related
- A lock from both may hash to the same slot
- The same `futex_hash_bucket::lock` is used for both
- Random memory addresses (ASLR) makes it not reproducible
- A report from 2024



And now?

- First attempts presented in 2016 at plumbers.
- Four main approaches.
- One was `FUTEX_ATTACH` with cookies instead of uaddr.
- Another was RCU-tree lookup
- ABI changes, glibc extensions, performance regressions



Reboot

- A task private hash per task (`futex_hash_bucket`)
- Scale up based on threads
- Use RCU for live time guarantees and `rcuref_t` for user tracking
- Resize means rehash and move waiter from old to new
- Performance was mostly okay except for micro benchmarks
- The atomic operation became visible.



Reboot continues

- Marked broken for v6.16, report from Chris Mason
- Replaced with per-CPU counter. Logic is similar to percpu-rwsem
- Per-CPU counter are used for reference tracking
- On resize, RCU lifetime guarantees are used to switch to global counter
- Once the counter goes to zero, the hash is replaced.



Reboot continues

- use per-CPU counter
- Switch to task wide atomic counter, wait RCU period
- Align per-CPU counter with the atomic counter
- Once the atomic counter goes to 0, next futex operation will replace the hash
- Operation requires to move possible waiters



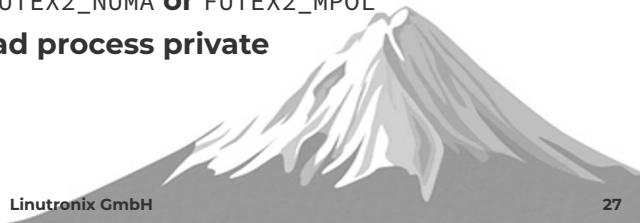
Memory location on NUMA

- **The global `struct futex_hash_bucket` is allocated interleaved during boot**
- **The data structure is used on random NUMA node**
- **HPC folks are not happy about it**
- **NUMA node local hash was preferred**
- **It has been worked on before ...**



MPOL and NUMA flags

- **With the previous changes, integration became easier**
- **In v6.7 futex2 syscall group was added.**
- **Instead** `sys_futex(, FUTEX_WAIT, …)`
- **There is now** `sys_futex_wait()`
- **The flags argument can be** `FUTEX2_NUMA` **or** `FUTEX2_MPOL`
- **It uses the global hash instead process private**



MPOL and NUMA flags

- `FUTEX2_NUMA` ⇒ **after uaddr there the NUMA is specified**
- **Node** `FUTEX_NO_NODE` ⇒ **use current and write back**
- `FUTEX2_MPOL` ⇒ **use memory policy**
- **Policy can be set with** `numa_set_mempolicy_home_node()` **(libnuma)**
- `FUTEX2_NUMA` **+** `FUTEX_NO_NODE` **+** `FUTEX2_MPOL` **can be use to check MPOL**
- **The hash is now used on the specified node.**



Lazy Preempt

- Merged (”sched: Add Lazy preemption model”) v6.13
- A preemption model next to None, Voluntary, Preemptible
- Delays scheduling requests to return-to-userland or next HZ tick
- Supported by arm64, loongarch, powerpc, riscv, s390, x86
- Performance is close to VOLUNTARY
- `cond_resched()` is a nop here.



Building block to avoid scheduling user space

- User space wants to avoid scheduling
- Make user space spin-locks more efficient
- Proposed in 2014 by Khalid Aziz
- Proposed again on top of lazy-preempt by Steven Rostedt in 2023
- Propposed by Prakash Sangappa in 2024, 2025
- Proposed again in 2025 Steven



Building block to avoid scheduling user space

- Thomas Gleixner started working on it
- v5 should be merged
- built on top of rseq interface
- approx 23% improvement on swingbench test
- Netflix's global-lock-bench shows an improvement starting with 64 CPUs
- Documentation/userspace-api/rseq.rst



mini example

```
rseq_register_current_thread();
prctl(PR_RSEQ_SLICE_EXTENSION, PR_RSEQ_SLICE_EXTENSION_SET,
      PR_RSEQ_SLICE_EXT_ENABLE, 0, 0);

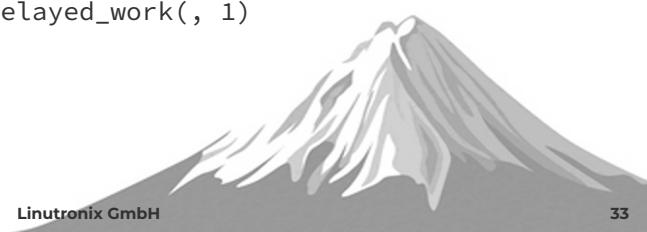
struct rseq_abi *rs = rseq_get_abi();
RSEQ_WRITE_ONCE(rs->slice_ctrl.request, 1);

/* critical section */

RSEQ_WRITE_ONCE(rs->slice_ctrl.request, 0);
if (RSEQ_READ_ONCE(rs->slice_ctrl.granted))
  syscall(__NR_rseq_slice_yield))
```

Workqueue and isolated CPUs

- `schedule_work()` ⇒ `queue_work(system_wq, )`
- `queue_work()` ⇒ `queue_work_on(WORK_CPU_UNBOUND,)`
- `system_wq = alloc_workqueue("events", 0, 0)` **/* flags */**
- **What is default behaviour for CPU placement?**
- **Is it equivalent to** `schedule_delayed_work(, 1)`
- **If not, should it be?**



Workqueue and isolated CPUs

- `queue_work(system_wq, )` **enqueues the work on current CPU if the workqueue is not** `WQ_UNBOUND`
- `schedule_delayed_work()` **enqueues a timer first**
- **The timer is enqueued either on housekeeping CPU or current**
- **Current CPU means** `add_timer_global()`
- **This means with** `NO_HZ` **the timer can expire on a remote CPU**
- **So it may or may not expire on the current CPU**



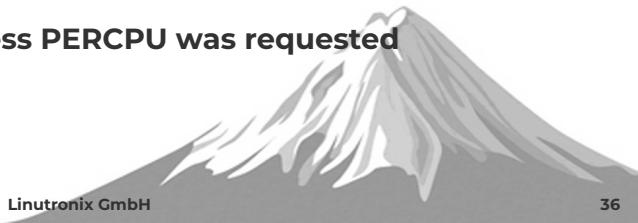
Workqueue and isolated CPUs

- On non-isolated CPUs probably nobody cares
- Most work probably don't require same CPU, or do they?
- For isolated workload it means, a syscall can start a work item.
- Issue reported. Upstream not happy about changing default behaviour.
- Marco Crivellari started refactoring the tree.



Workqueue API now

- `schedule_work()` ⇒ `queue_work(system_percpu_wq, )`
- `system_dfl_wq` **default unbound workqueue**
- **Marco moves users to either of the two workqueues**
- `system_wq`, `system_unbound_wq` , `WQ_UNBOUND` **could be removed once there are no more users.**
- **Makes unbound default unless PERCPU was requested**



Documentation

- Finally, some documentation.
- In kernel tree Documentation/core-api/real-time
- Covers how it should operate, differences vs !RT
- A list of items for new architectures
- More to come



Thank you for your attention



Special thanks to the Linux Foundation for supporting our efforts to bring PREEMPT_RT mainline.

bigeasy@linutronix.de